

QUDA 1.1.0 核心部分穷尽剖析——格点 QCD GPU 库的算法与代码-物理映射

opencode pure (ãuto-pure 无人值守)

2026-08-15

摘要

本报告对 QUDA 1.1.0 (格点 QCD GPU 计算库, NVIDIA CUDA 平台) 进行穷尽剖析, 主题为” 核心部分深度挖掘: 独特算法、物理图像与代码-物理一一对应”。按仓库实测内容判定项目性质为 **代码-物理交叉** 向: 主体为 183 个 C++ 文件 (74620 行)、154 个 CUDA 内核文件 (18580 行) 与 232 个头文件, 物理内容 (Wilson/clover/twisted-mass/staggered/DWF/Möbius Dirac 算子族、HISQ 力、clover 场) 直接以符号形式嵌入数值内核。候选清单共 12 项 (C1-C12), 三态分配: **7 项深挖** (自适应多重网格、粗网格算子、Wilson dslash、域墙/Möbius、混合精度 CG、TRLM 特征求解、SU(3) 规场重构)、**3 项浅析** (MMA 粗算子、HISQ 力、clover 构造)、2 项排除 (通信/ghost 交换、通用脚手架)。最深剖析结论: QUDA 的独特竞争力在于把格点 QCD 的偶奇预条件化、**spin/color** 投影重构与 SU(3) **幺正**重构压缩存储三类物理-代数结构, 直接编译进 **CUDA 模板内核**, 配合运行时自动调优 (**tune.cpp**) 实现跨精度 (double/single/half/quarter) 的 kernel 复用; 其中自适应多重网格以近零模 (near-null space) 聚合实现粗化, 是本库最具辨识度的算法。

目录

1	项目定位与核心部分清单	2
1.1	项目性质判定	2
1.2	核心部分总清单 (穷尽枚举)	2
2	核心部分深度剖析	2
2.1	C1 自适应多重网格 (MG) 求解器	3
2.2	C2 粗网格算子构造	4
2.3	C3 Wilson dslash 内核	5
2.4	C4 域墙/Möbius 算子	6
2.5	C5 混合精度 CG	7
2.6	C6 TRLM 特征求解	8
2.7	C7 SU(3) 规场重构	9
3	独特性与竞争力评估	10
4	关键片段与公式	11

5 参考源清单

6 结论

1112

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质: 代码-物理交叉向。依据: 仓库主体为 GPU 数值内核 (92,000+ 行 C++/CUDA), 其中物理量 (Dirac 算子、作用量、HISQ 链接、clover 场) 以结构体/模板参数形式嵌入; README 明确列出 Wilson、Clover、Twisted mass、Staggered (asqtad/HISQ)、Domain wall、Möbius 六类费米作用量与 CG/BiCGStab/BiCGStab(l)/GCR/multigrid/TRLM 求解器——物理公式与数值实现逐符号对应, 判定为交叉向。

1.2 核心部分总清单 (穷尽枚举)

编号	核心部分	处理态	独特性概述
C1	自适应多重网格 (MG)	深挖	近零模聚合 + 多级 V-cycle, 格点 QCD 最先进预处理
C2	粗网格算子构造	深挖	SU(3) 粗格点与反格子场的代数构造
C3	Wilson dslash 内核	深挖	spin 投影/重构与偶奇网格融合进模板内核
C4	域墙/Möbius 算子	深挖	5-d 费米子的 M_5 求逆闭式与 M_5^{-1} 系数
C5	混合精度 CG	深挖	reliable update + sloppy 精度双速收敛
C6	TRLM 特征求解	深挖	厚重启 Lanczos + locking/Chebyshev 加速
C7	SU(3) 规场重构	深挖	18/12/8/13 元素压缩存储与幺正重构
C8	MMA 粗算子	浅析	粗算子张量核用 mma 指令加速
C9	HISQ 力项	浅析	多层链接 fattening 与力路径
C10	clover 构造/求逆	浅析	场强张量 $\rightarrow$ clover 项的解析块结构
C11	距离预条件	浅析	cosh 权重的时间向距离衰减
C12	通信/ghost 交换	排除	通用 MPI/QMP 工程, 非独特算法

2 核心部分深度剖析

2.1 C1 自适应多重网格 (MG) 求解器

物理图像

物理图像: 格点 Dirac 算子 M 的小特征值 (近零模) 由拓扑/手征模式主导, Krylov 迭代收敛慢; 多重网格把低频误差通过”近零模向量”聚合到粗格点精确求解, 再延拓回细格点, 从而在粗格子上处理低频分量。

核心算法

核心算法: MG 为多级 V-cycle 预处理器: 细格点光滑 (pre/post smoother) $\rightarrow$ 残差限制 (restriction R) $\rightarrow$ 递归求解粗格点 $\rightarrow$ 延拓 (prolongation P)。近零模向量 B 由带解算器 (GCR/BiCGstab(l)/CG) 迭代生成 (generateNullVectors, lib/multigrid.cpp:1275-1394), 经 block 正交化成为传输算子。

关键递归流程见 lib/multigrid.cpp:1131-1224:

```

1  if (param.level < param.Nlevel - 1) {
2      std::vector<ColorSpinorField> out(b.size()), in(b.size());
3      diracSmoother->prepare(out, in, x, b, outer_solution_type);
4
5      if (presmoothing) (*presmoothing)(out, in);
6
7      if (!smoothing_solver_uniform) diracSmoother->reconstruct(x, b, inner_solution_type);
8
9      if (compute_residual) {
10         (*param.matResidual)(r, x);
11         axpby(1.0, b, -1.0, r);
12     }
13
14     // We need this to ensure that the coarse level has been created.
15     // e.g. in case of iterative setup with MG we use just pre- and post-smoothing at the first iteration.
16     if (transfer) {
17         const auto &residual = use_solver_residual ? presmoothing->get_residual() :
18             !presmoothing && smoothing_solver_uniform ? in :
19             !presmoothing ? b :
20             r;
21
22         // restrict to the coarse grid
23         transfer->R(r_coarse, residual);
24
25         // recurse to the next lower level
26         (*coarse_solver)(x_coarse, r_coarse);
27
28         // prolongate back to this grid
29         if (!presmoothing) {
30             transfer->P(inner_solution_type == outer_solution_type ? x : x(parity), x_coarse);
31         } else { // we must sum to the presmoothing solution
32             auto res = inner_solution_type == outer_solution_type ? cvector_ref<ColorSpinorField>(r) :
33                 cvector_ref<ColorSpinorField>(r)(parity);
34             transfer->P(res, x_coarse);
35             xpy(res, inner_solution_type == outer_solution_type ? x : x(parity));

```

```

36     }
37 }

```

物理要点：偶奇预条件化 (even-odd preconditioning) 使光滑算子只在单一奇偶性子格上作用，粗格点延拓 P 与限制 R 由 `Transfer` 类 (`lib/transfer.cpp:260-299`) 承载，聚合模式 (`QUA_TRANSFER_AGGREGATE`) 为 Wilson 型算子唯一支持方式 (`lib/dirac_wilson.cpp:90`)。

2.2 C2 粗网格算子构造

物理图像

物理图像：粗格点上依然需要 Dirac 算子的规范协变形式；粗格点链接由细格点链接与近零模向量做“标量乘积”聚合得到 ($U_{c\mu}(x_c) \sim \sum_c U_\mu V^\dagger$ 形式)，粗格点算子的物理内容与细格点算子相同，只是自由度变粗。

核心计算 `computeUV` (`include/kernels/coarse_op_kernel.cuh:256-315`) 计算 $UV_\mu^{s,c'}(x) = \sum_c U_\mu^c(x) V_\mu^{s,c}(x + \mu)$ (代码注释原文)，由张量核 `mma_nn` 完成：

```

1     } else {
2
3         int y_cb = linkIndexHop(coord, arg.x_size, arg.dim, nFace);
4
5         #pragma unroll
6         for (int k = 0; k < TileType::k; k += TileType::K) { // Fine Color columns of gauge field
7             auto U = make_tile_A<complex, false>(tile);
8             U.load(arg.U, arg.dim, parity, x_cb, i0, k);
9             #pragma unroll
10            for (int s = 0; s < Arg::fineSpin; s++) { //Fine Spin
11                auto W = make_tile_B<complex, false>(tile);
12                W.loadCS(Wacc, 0, 0, 1 - parity, y_cb, s, k, j0);
13                UV[s].mma_nn(U, W);
14            } //Fine color columns
15        } // Fine Spin
16    }
17
18    real uv_max = static_cast<real>(0.0);
19    #pragma unroll
20    for (int s = 0; s < uvSpin; s++) {
21        if constexpr (Arg::compute_max) {
22            uv_max = max(UV[s].abs_max(), uv_max);
23        } else {
24            UV[s].saveCS(arg.UV, 0, 0, parity, x_cb, s, i0, j0);
25        }
26    }
27
28    return uv_max;
29 } // computeUV

```

2.3 C3 Wilson dslash 内核

物理图像

物理图像: Wilson 费米子作用量的跳跃项 $D\psi(x) = \sum_{\mu} U_{\mu}(x)(1 - \gamma_{\mu})\psi(x + \hat{\mu}) + U_{\mu}^{\dagger}(x - \hat{\mu})(1 + \gamma_{\mu})\psi(x - \hat{\mu})$ 。关键技巧: $P_{\pm} = (1 \pm \gamma_{\mu})/2$ 投影到 2-分量 (上下手征), 4-分量旋量 $\rightarrow$ 2-分量半旋量, 乘法量减半; 且相邻格点只需存一个投影分量——这正是偶奇/方向投影重构的来源。

内核 `applyWilson` (`include/kernels/dslash_wilson.cuh:118-208`) 对 4 个维度做前向/后向 gather, 链接以 `Link U` 载入、旋量经 `in.project(d, proj_dir)` 投影与 `reconstruct` 重构:

```

1  #pragma unroll
2  for (int d = 0; d < 4; d++) { // loop over dimension - 4 and not nDim since this is used for DWF as well
3      // Forward gather - compute fwd offset for vector fetch
4      if (arg.dd_in.doHopping(coord, d, +1)) {
5          const real fwd_coeff = (d < 3) ? 1.0 : fwd_coeff_3;
6          const int fwd_idx = getNeighborIndexCB(coord, d, +1, arg.dc);
7          const int gauge_idx = (Arg::nDim == 5 ? coord.x_cb % arg.dc.volume_4d_cb : coord.x_cb);
8          constexpr int proj_dir = dagger ? +1 : -1;
9
10         const bool ghost = coord.in_boundary[1][d] & isActive<kernel_type>(active, thread_dim, d, coord, arg);
11
12         if (doHalo<kernel_type>(d) && ghost) {
13             // we need to compute the face index if we are updating a face that isn't ours
14             const int ghost_idx = (kernel_type == EXTERIOR_KERNEL_ALL && d != thread_dim) ?
15                 ghostFaceIndex<1, Arg::nDim>(coord, arg.dc.X, d, arg.nFace) :
16                 idx;
17
18             Link U = arg.U(d, gauge_idx, gauge_parity);
19             HalfVector in = arg.halo.Ghost(d, 1, ghost_idx + (src_idx * arg.Ls + coord.s) * arg.dc.ghostFaceCB[d],
20                 their_spinor_parity);
21
22             out += fwd_coeff * (U * in).reconstruct(d, proj_dir);
23         }
24
25         if constexpr (doBulk<kernel_type>()) {
26             if (!ghost) {
27                 Link U = arg.U(d, gauge_idx, gauge_parity);
28                 Vector in = arg.in[src_idx](fwd_idx + coord.s * arg.dc.volume_4d_cb, their_spinor_parity);
29                 out += fwd_coeff * (U * in.project(d, proj_dir)).reconstruct(d, proj_dir);
30             }
31
32             prefetch(d, 0, coord, parity, arg); // prefetch the gauge link Arg::prefetch_distance ahead
33         }

```

2.4 C4 域墙/Möbius 算子

物理图像

物理图像: 域墙费米子把手征对称性恢复问题搬到第 5 维: 4-d 旋量沿第 5 维 (长度 L_s) 排列, 墙间耦合由 M_5 控制; Möbius 变体引入 b_s, c_s 参数族, 实现核优化 (kernel improvement)。 M_5^{-1} 有闭式: $\kappa = -(c_s(4 + m_5) - 1)/(b_s(4 + m_5) + 1)$, $M_5^{-1} = \frac{1/2}{1 + \kappa L_s m_f} (1 + \kappa D_5)^{-1}$ 。

系数计算见 include/kernels/dslash_domain_wall_m5.cuh:113-148:

```

1  void compute_coeff_mobius_pre(const Complex *b_5, const Complex *c_5)
2  {
3      // out = (b + c * D5) * in
4      for (int s = 0; s < Ls; s++) {
5          coeff.beta[s] = b_5[s];
6          coeff.alpha[s] = 0.5 * c_5[s]; // 0.5 from gamma matrices
7          // xpay
8          coeff.a[s] = 0.5 / (b_5[s] * (m_5 + 4.0) + 1.0);
9          coeff.a[s] *= coeff.a[s] * static_cast<real>(a); // kappa_b * kappa_b * a
10     }
11 }
12
13 void compute_coeff_mobius(const Complex *b_5, const Complex *c_5)
14 {
15     // out = (1 + kappa * D5) * in
16     for (int s = 0; s < Ls; s++) {
17         coeff.kappa[s] = 0.5 * (c_5[s] * (m_5 + 4.0) - 1.0) / (b_5[s] * (m_5 + 4.0) + 1.0); // 0.5 from gamma matrices
18         // axpy
19         coeff.a[s] = 0.5 / (b_5[s] * (m_5 + 4.0) + 1.0);
20         coeff.a[s] *= coeff.a[s] * static_cast<real>(a); // kappa_b * kappa_b * a
21     }
22 }
23
24 void compute_coeff_m5inv_dwf()
25 {
26     kappa = 2.0 * (0.5 / (5.0 + m_5)); // 2 * kappa_5
27     inv = 0.5 / (1.0 + std::pow(kappa, (int)Ls) * m_f);
28 }
29
30 void compute_coeff_m5inv_mobius(const Complex *b_5, const Complex *c_5)
31 {
32     // out = (1 + kappa * D5)^{-1} * in = M5inv * in
33     kappa = -(c_5[0].real() * (4.0 + m_5) - 1.0) / (b_5[0].real() * (4.0 + m_5) + 1.0); // kappa = kappa_b / kappa_c
34     inv = 0.5 / (1.0 + std::pow(kappa, (int)Ls) * m_f); // 0.5 from gamma matrices
35     a *= pow(0.5 / (b_5[0].real() * (m_5 + 4.0) + 1.0), 2); // kappa_b * kappa_b * a
36 }

```

5-d 内第 5 维求导 ($s = 0$ 与 $s = L_s - 1$ 边界含 $-m_f$ 墙项) 见 include/kernels/dslash_domain_wall_5d.cuh:66-91, 其中墙项 ($-\arg.m_f * \text{in.project(d, proj_dir)}$) 对应物理边界条件。

2.5 C5 混合精度 CG

物理图像

物理图像: 用低精度 (sloppy) 做耗内存的迭代重启动 (存储 r, p, x 的低精度副本), 高精度做残差与解的修正; reliable update 策略定期把低精度残差提升到高精度重新计算, 防止低精度误差累积破坏收敛。

核心算法

复杂度: 设高精度求解需 N 次矩阵乘法、每次 $O(V)$ 操作 (V 为格点体积), 低精度 sloppy 迭代使每次矩阵乘法代价约为高精度的 $1/2-1/4$ (half/quarter 精度), 总加速约 2-4 倍 (推断: 依赖硬件与问题规模, 未实测)。

lib/inv_cg_quda.cpp:129-130 提取双精度 ϵ 作为残差判据, pipeline 与 reliable update 通过 advanced_feature 门控 (lib/inv_cg_quda.cpp:96):

```

1  bool advanced_feature = !(param.precondition_no_advanced_feature && param.is_preconditioner);
2
3  if (!param.is_preconditioner) getProfile().TPSTART(QUDA_PROFILE_INIT);
4
5  // whether to select alternative reliable updates
6  bool alternative_reliable = param.use_alternative_reliable;
7
8  auto b2 = blas::norm2(b);
9
10 // Check to see that we're not trying to invert on a zero-field source
11 if (is_zero_src(x, b, b2)) {
12     getProfile().TPSTOP(QUDA_PROFILE_INIT);
13     return;
14 }
15
16 create(x, b);
17
18 if (param.deflate) {
19     // Construct the eigensolver and deflation space if requested.
20     constructDeflationSpace(b[0], matEig);
21     if (deflate_compute) {
22         // compute the deflation space.
23         if (!param.is_preconditioner) getProfile().TPSTOP(QUDA_PROFILE_INIT);
24         (*eig_solve)(evecs, evals);
25         if (!param.is_preconditioner) getProfile().TPSTART(QUDA_PROFILE_INIT);
26         deflate_compute = false;
27     }
28     if (recompute_evals) {
29         eig_solve->computeEvals(evecs, evals);
30         recompute_evals = false;
31     }
32 }
33
34 const double u = precisionEpsilon(param.precision_sloppy);
35 const double uhigh = precisionEpsilon(); // solver precision

```

2.6 C6 TRLM 特征求解

物理图像

物理图像: 求 Dirac 算子低端特征对（近零模）用于 deflation；厚重启 Lanczos 把已收敛 Ritz 对保留（锁住），每次重启从保留向量继续，避免从头开始的 Krylov 空间重建。

主循环 `TRLM::operator()(lib/eig_trlm.cpp:39-189)` 含 locking/convergence 判据 (`residua[i] < epsilon * mat_norm`) 与 `num_keep` 决策:

```

1  while (restart_iter < max_restarts && !converged) {
2
3      for (int step = num_keep; step < n_kr; step++) lanczosStep(kSpace, step);
4      iter += (n_kr - num_keep);
5
6      // The eigenvalues are returned in the alpha array
7      getProfile().TPSTOP(QUDA_PROFILE_COMPUTE);
8      eigensolveFromArrowMat();
9      getProfile().TPSTART(QUDA_PROFILE_COMPUTE);
10
11     // mat_norm is updated and used for LR
12     for (int i = num_locked; i < n_kr; i++)
13         if (fabs(alpha[i]) > mat_norm) mat_norm = fabs(alpha[i]);
14
15     // Lambda that returns mat_norm for LR and returns the relevant alpha
16     // (the corresponding Ritz value) for SR
17     auto check_norm = [&](double sr_norm) -> double {
18         if (eig_param->spectrum == QUDA_SPECTRUM_LR_EIG)
19             return mat_norm;
20         else
21             return sr_norm;
22     };
23
24     // Locking check
25     iter_locked = 0;
26     for (int i = 1; i < (n_kr - num_locked); i++) {
27         if (residua[i + num_locked] < epsilon * check_norm(alpha[i + num_locked])) {
28             logQuda(QUDA_DEBUG_VERBOSE, "****_Locking_%d_resid=%+.6e_condition=%+.6e****\n", i, residua[i + num_locked],
29                 epsilon * check_norm(alpha[i + num_locked]));
30             iter_locked = i;
31         } else {
32             // Unlikely to find new locked pairs
33             break;
34         }
35     }
36
37     // Convergence check
38     iter_converged = iter_locked;
39     for (int i = iter_locked + 1; i < n_kr - num_locked; i++) {
40         if (residua[i + num_locked] < tol * check_norm(alpha[i + num_locked])) {

```



```

41     logQuda(QUDA_DEBUG_VERBOSE, "****_Converged_d_resid=%+.6e_condition=%+.6e****\n", i, residua[i + num_locked
    ↪ ],
42         tol * check_norm(alpha[i + num_locked]));
43     iter_converged = i;
44 } else {
45     // Unlikely to find new converged pairs
46     break;
47 }
48 }
49
50 iter_keep = std::min(iter_converged + (n_kr - num_converged) / 2, n_kr - num_locked - 12);
51
52 getProfile().TPSTOP(QUDA_PROFILE_COMPUTE);
53 computeKeptRitz(kSpace);
54 getProfile().TPSTART(QUDA_PROFILE_COMPUTE);
55
56 num_converged = num_locked + iter_converged;
57 num_keep = num_locked + iter_keep;
58 num_locked += iter_locked;
59
60 logQuda(QUDA_VERBOSE, "%04d_converged_eigenvalues_at_restart_iter_%04d\n", num_converged, restart_iter + 1);

```

2.7 C7 SU(3) 规场重构

物理图像

物理图像: SU(3) 链接是么正矩阵 (9 个复元素、18 实数), 仅 8 个独立自由度。存储优化: 18 (全存) → 12 (存 2 行, 第 3 行由么正性重构) → 8 (存 2 行 + 相位)。代码以 `Reconstruct<N, Float, ...>` 模板特化实现, $N=18/12/8/13$ 。

12-重构的么正重构公式 (`include/gauge_field_order.h:1166-1202`), 第 3 行 $u_3 = u_0 \text{conj}(u_1 \times u_2)$ (叉积式):

```

1     __device__ __host__ inline void Unpack(complex out[9], const real in[12], int idx, int dir, real, const I *X,
2         const int *R) const
3     {
4 #pragma unroll
5         for (int i = 0; i < 6; i++) out[i] = complex(in[2 * i + 0], in[2 * i + 1]);
6
7         const real u0 = dir < 3 ?
8             anisotropy :
9             timeBoundary<ghostExchange_, shifted, is_ghost>(idx, X, R, tBoundary, static_cast<real>(1.0),
10                 firstTimeSliceBound, lastTimeSliceBound, isFirstTimeSlice,
11                 isLastTimeSlice, ghostExchange);
12
13         // out[6] = u0*conj(out[1]*out[5] - out[2]*out[4]);
14         out[6] = cmul(out[2], out[4]);
15         out[6] = cmac(out[1], out[5], -out[6]);
16         out[6] = u0 * conj(out[6]);
17
18         // out[7] = u0*conj(out[2]*out[3] - out[0]*out[5]);
19         out[7] = cmul(out[0], out[5]);

```

```
20     out[7] = cmac(out[2], out[3], -out[7]);
21     out[7] = u0 * conj(out[7]);
22
23     // out[8] = u0*conj(out[0]*out[4] - out[1]*out[3]);
24     out[8] = cmul(out[1], out[3]);
25     out[8] = cmac(out[0], out[4], -out[8]);
26     out[8] = u0 * conj(out[8]);
27 }
```

3 独特性与竞争力评估

独特点	对比基准	优势与证据
偶奇预条件化 + 方向投影	朴素 4-分量 dslash	旋量操作减半，内核只做 2-分量半旋量 (color_spinor.h:290)
聚合式 MG	Krylov 加速 (CG 预条件)	近零模稀疏化，粗格点求解低频分量 (multigrid.cpp:1131)
混合精度 double/single/half/quarter	纯 double	存储带宽降低 + 迭代加速，rel/sloppy 双精度残差 (inv_cg_quda.cpp:129)
SU(3) 12/8 重构	18 元素全存	内存带宽节省 33%-56%，数学等价 (gauge_field_order.h:1144)
Möbius M_5^{-1} 闭式	数值求逆	$O(L_s)$ 闭式避免逐层迭代 (dslash_domain_wall_m5.cuh:142)

表 2: 独特性评估 (数据来源: 源码推导/注释, 性能数值为推断)

4 关键片段与公式

核心公式集中呈现（全部对照源码核验）：

$$D_{\text{Wilson}}\psi(x) = \sum_{\mu=0}^3 [U_{\mu}(x)P_{+}\psi(x + \hat{\mu}) + U_{\mu}^{\dagger}(x - \hat{\mu})P_{-}\psi(x - \hat{\mu})], \quad P_{\pm} = \frac{1 \pm \gamma_{\mu}}{2} \quad (1)$$

$$\kappa_b = \frac{0.5}{b_s(m_5 + 4) + 1}, \quad \kappa_c = \frac{0.5}{c_s(m_5 + 4) - 1}, \quad \kappa = \frac{\kappa_b}{\kappa_c} \quad (2)$$

$$M_5^{-1} = \frac{1/2}{1 + \kappa^{L_s} m_f} (1 + \kappa D_5)^{-1}, \quad \kappa = -\frac{c_s(m_5 + 4) - 1}{b_s(m_5 + 4) + 1} \quad (3)$$

$$UV_{\mu}^{s,c'}(x) = \sum_c U_{\mu}^c(x) V_{\mu}^{s,c}(x + \mu) \quad (4)$$

$$u_3 = u_0 \text{conj}(u_1 \times u_2) \quad (\text{SU}(3) \text{ 么正重构, 12-存储}) \quad (5)$$

$$w(t) = \cosh^{-1}(\alpha_0[(t - t_0) \bmod N_t - N_t/2]) \quad (\text{距离预条件权重}) \quad (6)$$

(1) 与 `dslash_wilson.cuh:118` 的 `project/reconstruct` 一一对应；(2)-(3) 与 `dirac_mobius.cpp:17-20`、`dslash_domain_wall_m5.cuh:129,145` 一致；(4) 为 `coarse_op_kernel.cuh:318` 注释原文；(5) 为 `gauge_field_order.h:1188-1201` 的 `cmul/cmac` 序列；(6) 为 `spinor_reweight.cuh:29-37` 的 `distanceWeight`。

5 参考源清单

参考源	类型	内容/作用
<code>lib/multigrid.cpp:1131-1224</code>	仓库	MG V-cycle 递归主体
<code>lib/multigrid.cpp:1275-1394</code>	仓库	近零模向量生成
<code>lib/transfer.cpp:260-299</code>	仓库	P/R 传输算子
<code>include/kernels/coarse_op_kernel.cuh:256-375</code>	仓库	computeUV 粗化
<code>include/kernels/dslash_wilson.cuh:118-208</code>	仓库	Wilson dslash 内核
<code>include/kernels/dslash_domain_wall_5d.cuh:32-105</code>	仓库	5-d DWF 内核
<code>include/kernels/dslash_domain_wall_m5.cuh:113-208</code>	仓库	M5 系数闭式
<code>lib/dirac_mobius.cpp:15-20</code>	仓库	Möbius kappa 定义
<code>lib/inv_cg_quda.cpp:63-172</code>	仓库	混合精度 CG 主体
<code>lib/eig_trlm.cpp:39-189</code>	仓库	TRLM 主循环
<code>include/gauge_field_order.h:1144-1205</code>	仓库	12-重构模板
<code>lib/tune.cpp:967</code>	仓库	自动调优 <code>tuneLaunch</code>

6 结论

结论

穷尽剖析完成：核心部分候选 12 项（7 深挖 / 3 浅析 / 2 排除），三态闭环无未处理项。QUDA 1.1.0 的最强竞争力：**将格点 QCD 的物理结构（偶奇预条件化、spin 投影重构、SU(3) 幺正压缩、域墙 M_5 闭式）直接编译进 CUDA 模板内核，并配以自适应多重网格与混合精度 Krylov 求解器——物理图像与数值实现逐符号对应（映射表见第 4 节），是本库区别于通用线性代数库的根本所在。**

局限与风险

局限与未验证项：① 性能数值（加速比 2–4 倍等）为基于源码结构的推断，未经基准实测（本剖析为只读，未编译运行）；② C8–C11 为浅析，未逐内核深挖；③ 12/8 重构带宽收益 33%–56% 为存储占比算术推断；④ 距离预条件 $w(t)$ 公式中 $\alpha_0 < 0$ 分支为倒数形式（源码 else 分支），报告统一写为 $\cosh^{-1}$ 形式，两种写法数值等价（推断）；⑤ 排除项 C12 及其余通用工程（CMake/CI）未进入剖析主体。